Introduksjon til Unit testing med JUnit 5

Om testing av programvare

Det finnes ulike typer testing innenfor software industrien, det kan nevnes

- Akseptansetester
  - har kunden fått det han ønsket og var blitt lovet?
- Integrasjonstester
  - fungerer ny modul (del av softwaren som utvikles) sammen med andre moduler; - er levert programvare klar for en systemtest?
- Systemtester
  - fungerer levert programvare sammen med helheten med virkelig belastning, sikkerhet, pålitelighet, ytelse?
- Enhetstester (unit testing)
  - vi tester på klasse / metodenivå

JUnit

Et testrammeverk for Javaprogrammer.
Basert på Xunit, en kollektiv benevnelse på «unit testing frameworks» basert på Smalltalks SUnit, designet av Kent Beck i 1998.
SUnit portet til Java av Kent Beck og Erich Gamma.

Test runner

Et program som kjører JUnit tester. Følger med Eclipse / IntelliJ slik at vi kan kjøre testene (Run As Junit test case..)

Hva er en enhetstest?

Du vil finne at begrepet enhetstesting / unit testing har mange ulike «definisjoner». Stort sett vil du finne at dette beskrives omtrent som «den minste komponent som kan testes», hvilket gjerne da er en metode. Andre vil si at en komponent er en klasse med sine metoder. En av de største guruene innenfor unit testing. Martin Fowler, synser rundt dette her:
https://martinfowler.com/bliki/UnitTest.html

«there is a notion that unit tests are low-level, focusing on a small part of the software system. Secondly unit tests are usually written these days by the programmers themselves using their regular tools - the only difference being the use of some sort of unit testing framework. Thirdly unit tests are expected to be significantly faster than other kinds of tests.

So there's some common elements, but there are also differences. One difference is what people consider to be a unit. Object-oriented design tends to treat a class as the unit, procedural or functional approaches might consider a single function as a unit. But really it's a situational thing - the team decides what makes sense to be a unit for the purposes of their understanding of the system and its testing. Although I start with the notion of the unit being a class, I often take a bunch of closely related classes and treat them as a single unit. Rarely I might take a subset of methods in a class as a unit. However you define it doesn't really matter.»

Et konkret eksempel

For å forstå behovet for enhetstesting kan vi ta for oss en enkel Java klasse `Clock` med metodene `setTime`, `incrementSeconds` og `getTimeAsString`

```
public class Clock {
	private int hours = 0;
	private int minutes = 0;
	private int seconds = 0;

	public  void setTime(int h, int m, int s) {
		hours = h;
		minutes = m;
		seconds = s;
	}

	public int incrementSeconds() {
		seconds++;
		if (seconds == 60) {
			seconds = 0;
			minutes++;
		}
		if (minutes == 60) {
			minutes = 0;
			hours++;
		}
		if (hours == 24)
			hours = minutes = seconds = 0;

		return (hours * 60 * 60) + (minutes * 60) + seconds;
	}

	public int incrementMinutes() {
		minutes++;
		if (minutes == 60) {
			minutes = 0;
			hours++;
		}
		if (hours == 24)
			hours = minutes = seconds = 0;

		return (hours * 60 * 60) + (minutes * 60) + seconds;
	}

	public int incrementHours() {
		hours++;

		if (hours == 24)
			hours = minutes = seconds = 0;

		return (hours * 60 * 60) + (minutes * 60) + seconds;
	}

	public String getTimeAsString() {
		return String.format("%02d:%02d:%02d", hours, minutes, seconds);
	}
}
```

For å teste koden vil det være naturlig å lage en `main()` metode i `Clock`, eller bedre, en egen klasse som inneholder en `main()` som påkaller metoden i `Clock`.

Denne test koden vil kunne være:

```
public class TestClock {
	public static void main(String[] args) {
		Clock myClock = new Clock();
		myClock.setTime(0,0,0);
		if (myClock.getTimeAsString().equals("00:00:00"))
			System.out.println("Clock is correct");
		else
			System.out.println("Clock is incorrect: " +
myClock.getTimeAsString());

		int result = myClock.incrementSeconds();
		if (result == 1)
			System.out.println("Clock is correct");

		myClock.setTime(23,50,59);
		System.out.println(myClock.getTimeAsString());
		myClock.incrementSeconds();
		System.out.println(myClock.getTimeAsString());
	}
}
```

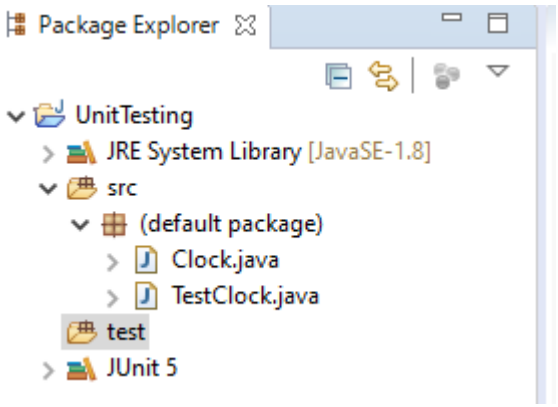
Testingen består i en sammenligning av strenger, en visuell observasjon av utskriften fra `getTimeAsString`, samt en test på at sekund er inkrementert korrekt fra 0.

Det er flere grensetilfeller som ikke er testet, og en slik testing som over er som regel ikke systematisk i sin natur. Denne koden lages som regel når `Clock` lages; - den fjernes på et tidspunkt og tas ikke fram igjen, selv om vi går inn og endrer koden.

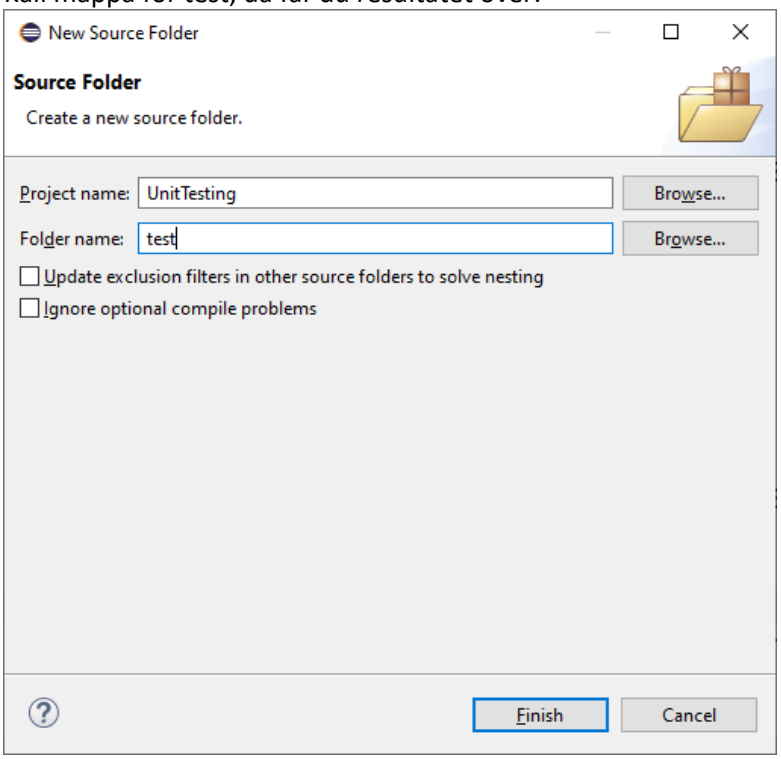
Unit testing er en måte å systematisere testingen på. Vi bruker et ***rammeverk*** for testing for å lage enkle, raske tester som kjøres HVER GANG vi har gjort en endring i en eller annen metode i vår test suite (de klasser og metoder en har bestemt seg for å teste under ett).
Vi lager tester for de metodene som har public grensesnitt, og vi tester både for «solskinnsdata» («normale data»), og for grensetilfeller. Alle feil skal kunne avsløres av kode i testen, vi stoler ikke på visuell observasjon (utskrift).

Junit 5 er tilgjengelig fra de siste Eclipse og IntelliJ versjoner. Når vi lager tester, kan det være lurt å plassere disse i en source code mappe som vi kaller ***test***, som ligger på samme nivå som mappa der kildekoden ligger (src).

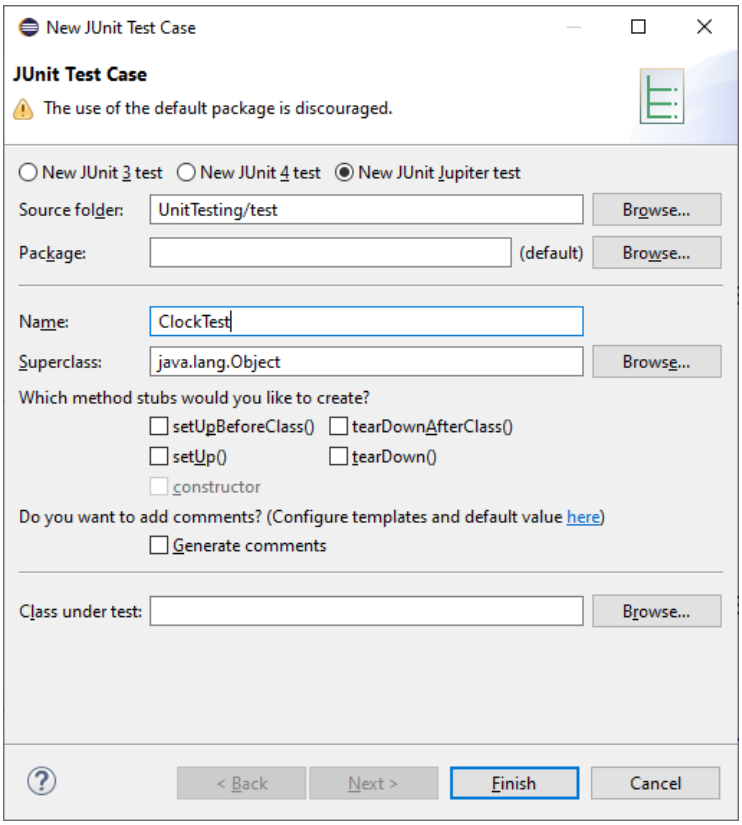
Mappa test i bildet under er laget ved at du høyreklikker på prosjektet ditt (her UnitTesting), og velger New Source Folder.



Kall mappa for test, da får du resultatet over:



Klikk på test mappa og velg *New Junit Test Case*:



NB! Oppgi klassen du skal teste i feltet «Class under test» (ikke fylt her, men du skal fylle ut…)
Det kan hende du blir spurt om å inkludere Jupiter (Jupiter er den 5. planeten sett fra sola, JUnit 5…), svar i så fall bekreftende.
Du får laget en stub med kode:

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class ClockTest {

	@Test
	void test() {
		fail("Not yet implemented");
	}

}
```

Tester lager vi basert på såkalte ***assertions*** (i koden nedenfor er `assertEquals(..)` et eksempel på en assertion). En assertion går ut på at vi forventer en gitt respons på en gitt input. I prinsippet tester vi på at den returverdien vi får fra en metode er den vi forventer, gitt den input vi gir.

*Merk at i **testdrevet utvikling** så skriver vi testene **først**, deretter begynner vi å skrive koden for metoden som skal passere testen.*

Vår første test går ut på å sjekke om antall sekunder som returneres fra `incrementSeconds` er lik 1 etter inkrementering.

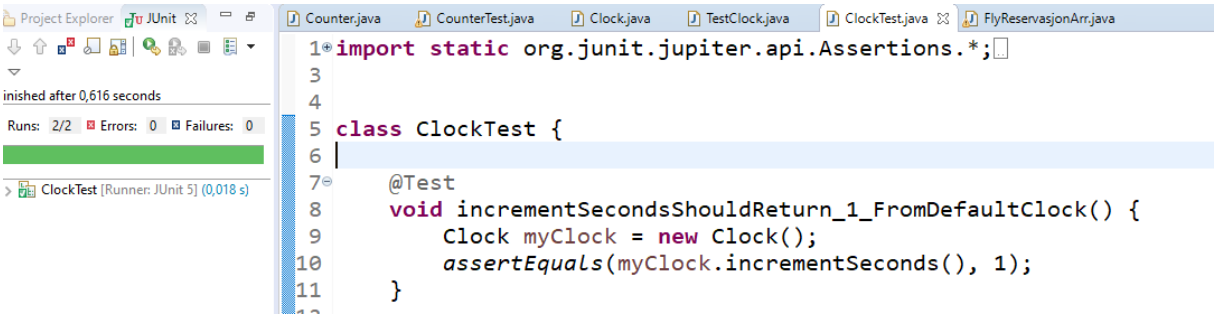
Vi bruker annotasjonen **`@Test`** for å fortelle at JUnit5 skal oppfatte etterfølgende kode som en test.

Testen vil kunne implementeres slik:

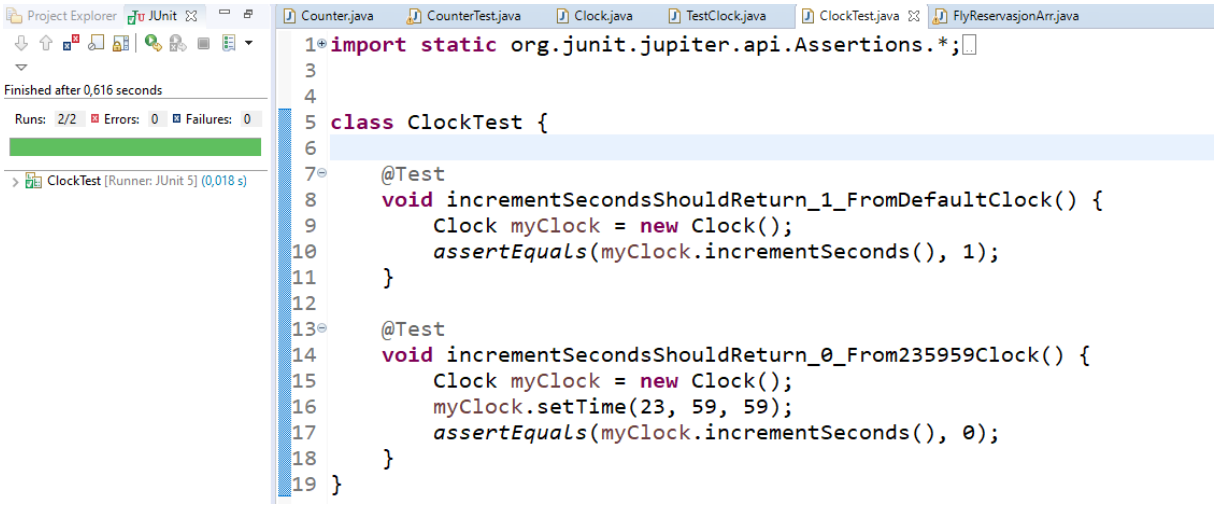
```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class ClockTest {

	@Test
	void test() {
		Clock myClock = new Clock();
		assertEquals(Clock.incrementSeconds(), 1);
	}

}
```

Vi kjører den som en test ved å velge `ClockTest` klassen, og velger *Run As Test*.

Dersom testen passerer, får vi opp en kvittering på at den har passert:



Når vi lager tester, skal vi gi de et navn som forteller hvilken metode vi tester, og hvilken retur vi forventer gitt en input. Jeg vil endre navnet til f eks slik som vist nedenfor. Det er også lagt inn en ny test som sjekker et viktig grensetilfelle; - at klokka er 23:59:59 og skal tippe rundt til 0:



I unit testeing er det et begrep som heter AAA, som står for

Arrange (prepare the test data)
Act (do what you are testing)
Assert (check the results)

I testene som er vist over har vi bruke *act* og *assert*. *Act* er at vi utfører et kall mot metoden vi ønsker å teste, *assert* er testen på resultatet. Dersom klassen vi tester ikke er static, må vi huske på å lage et objekt før vi kaller dens metoder. I stedet for at hver test lager et slikt objekt, kan vi bruke annotasjonen **@BeforeEach**, som vil kjøres foran hver test. Vi oppretter heller objektet i **@BeforeEach** enn i hver enkelt test. Dette er *Arrange* delen av testen; å passe på at vi har gjort en del forberedelser før vi kjører testen. Denne forberedelsen kan selvsagt være mer komplisert enn bare å skape et objekt.

I eksempelet under har vi en klasse Counter som vi ønsker å teste. Counter har to instansmetoder (en instansmetode husker du er en metode som må kalles fra et objekt) som vi ønsker å teste. I hver test er det en forutsetning at et objekt er opprettet. I stedet for å opprette objektet i hver test, kan vi gjøre dette i en metode som er laget under annotasjonen @BeforeEach:

```
// klassen vi skal teste
public class Counter {
      int count = 0;

      public int increment() {
            if (count < Integer.MAX_VALUE)
                  count += 1;
            return count;
      }

      public int decrement() {
            if (count > 0)
              count -= 1;
            return count;
      }
}

// testklassen
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;

public class CounterTest {
      Counter counter;

      @BeforeEach
      public void setup() {
            counter = new Counter();
      }

      @Test
      public void incrementFromZeroShouldIncrementToOne() {
            assertEquals(1,counter.increment());
      }

      @Test
      public void decrementFromZeroShouldNotDecrement() {
            assertEquals(0,counter.decrement());
      }
```

```
	@Test
	public void decrementAfterIncrementShouldReturnZero() {
		assertEquals(1,counter.increment());
		assertEquals(0,counter.decrement());
	}
}
```

Eksempler på ulike typer assertions relevant for dette kurset (hentet fra https://junit.org/junit5/docs/current/user-guide/)

JUnit 5 kept many of the assertion methods of JUnit 4 while adding few new ones that take advantage of the Java 8 support.

Also in this version of the library, assertions are available for all primitive types, *Objects,* and arrays (either of primitives or Objects).

The order of the parameters of the assertions changed, moving the output message parameter as the last parameter.

assertArrayEquals

The *assertArrayEquals* assertion verifies that the expected and the actual arrays are equals:

```
@Test
public void whenAssertingArraysEquality_thenEqual() {
    char[] expected = { 'J', 'u', 'p', 'i', 't', 'e', 'r' };
    char[] actual = "Jupiter".toCharArray();

    assertArrayEquals(expected, actual, "Arrays should be equal");
}
```

If the arrays aren't equal, the message "*Arrays should be equal*" will be displayed as output.

assertEquals

In case we want to assert that two *floats* are equals, we can use the simple *assertEquals* assertion:

```
@Test
public void whenAssertingEquality_thenEqual() {
    float square = 2 * 2;
    float rectangle = 2 * 2;

    assertEquals(square, rectangle);
}
```

However, if we want to assert that the actual value differs by a predefined delta from the expected value, we can still use the *assertEquals* but we have to pass the delta value as the third parameter:

```
@Test
public void whenAssertingEqualityWithDelta_thenEqual() {
    float square = 2 * 2;
    float rectangle = 3 * 2;
    float delta = 2;

    assertEquals(square, rectangle, delta);
}
```

assertTrue and assertFalse

With the *assertTrue* assertion, it's possible to verify the supplied conditions are *true*:

```
@Test
public void whenAssertingConditions_thenVerified() {
    assertTrue(5 > 4, "5 is greater the 4");
    assertTrue(null == null, "null is equal to null");
}
```

assertLinesMatch

The *assertLinesMatch* asserts that the expected list of *String* matches the actual list.

This method differs from the *assertEquals* and *assertIterableEquals* since, for each pair of expected and actual lines, it performs this algorithm:

check if the expected line is equal to the actual one. If yes it continues with the next pair

treat the expected line as a regular expression and performs a check with the *String.matches()* method. If yes it continues with the next pair

check if the expected line is a fast-forward marker. If yes apply fast-forward and repeat the algorithm from the step 1

Let's see how we can use this assertion to assert that two lists of *String* have matching lines:

```
@Test
public void whenAssertingEqualityListOfStrings_thenEqual() {
    List<String> expected = asList("Java", "\\d+", "JUnit");
    List<String> actual = asList("Java", "11", "JUnit");

    assertLinesMatch(expected, actual);
}
```

assertNotEquals

Complementary to the *assertEquals*, the *assertNotEquals* assertion asserts that the expected and the actual values aren't equal:

```
@Test
public void whenAssertingEquality_thenNotEqual() {
    Integer value = 5; // result of an algorithm

    assertNotEquals(0, value, "The result cannot be 0");
}
```

If both are *null*, the assertion fails.

Annotasjoner (JUnit 5, flere av disse het noe annet i tidligere versjoner)

@Test
Plasseres foran hver metode som skal være en test. JUnit vil kjenne igjen og kjøre denne som en test. Husk at metoder *utføres i tilfeldig rekkefølge*, så det er en dårlig ide å anta noe om tilstanden til det du tester.

@Disabled
Plasseres foran en @Test dersom aktuell test ikke skal kjøres

@BeforeEach
Plasseres foran en metode som skal kjøres *før enhver test*. Slike metoder er en del av arrange delen av en test; sette opp objekter eller testdata som skal brukes i flere tester

@BeforeAll
Et metode med denne annotasjonen kjøres *en* gang, og før testene utføres.

@AfterEach
Utføres etter alle tester. Lite brukt.

@AfterAll
Utføres en gang etter at alle tester har kjørt. Lite brukt.

@ParameterizedTest
Markerer at en test er en parameterisert test, dvs den gir inn mange data og tester på disse.

Parameteriserte tester

Dette er en test hvor vi kan sende inn en liste av verdier, samt en tilhørende svarverdi.
Vi markerer at testen er parameterisert ved å angi annotasjonen `@ParameterizedTest`.
Deretter kommer vi med en liste av inputverdier, evt også forventede returverdier, se annotasjoner som brukes under. Vi må tenke oss at den testmetoden vi lager vil bli kjørt like mange ganger som det er elementer i `@ValueSource` eller `@CsvSource`. Under vises bare noen veldig enkle eksempler på hvordan en kan bruke parameteriserte tester.

Nødvendige imports i eksemplene under:

```
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
```

@ValueSource

Kan kun sende en liste av verdier, ikke oppgi forventede returverdier.
Eksempel, vi tester en metode `checkPassword()`, hvor alle inngitte verdier skal være ulovlige.

```
@ParameterizedTest
@ValueSource(strings = {"12abcDEFGH","abcdefghijklm","abc123","abc123#+"})
public void checkPassword_AllShouldReturnFalse(String input) {
      assertFalse(Password.checkPassword(input));
```

```
}

Et annet eksempel med @ValueSource:
@ParameterizedTest
@ValueSource(ints = {1, 3, 5, -3, 15, Integer.MAX_VALUE})
void isOdd_ShouldReturnTrueForOddNumbers(int number) {
    assertTrue(Numbers.isOdd(number));
}
```

@CsvSource

Med denne kan vi sende inn en liste av inputverdier OG forventet resultat:

```
@ParameterizedTest
@CsvSource({"1,1","2,3"})
public void increment_incrementWithArgument(int input, int expected) {
      assertEquals(expected,counter.increment(input));
}
```

Det finnes også en annotasjon `@CsvFileSource` som gjør at enkan lese testdata fra fil.

Hamcrest

Kommer

Her er lenker til en del leseverdige artikler om enhetsteting
https://www.toptal.com/qa/how-to-write-testable-code-and-why-it-matters